

Parcial - tema2

Nota: 8.8 / 10.0 (APROBADO)

puntaje ej1: 2
puntaje ej2: 1.75
puntaje ej3: 2.5
puntaje ej4: 2
puntaje ej5: 0.0
puntaje ej6: 0.5

Datos del alumno

Nombre: Giuly Belo

DNI: ☹ ☹ ☹ ☹ ☹ ☹

LU: ☹ ☹ ☹ ☹ ☹ ☹

Maquina: 37-06

Enunciado

Parcial Python - Tema 2

Importante

- El parcial se aprueba con 6 puntos
- Template de funciones a implementar [acá](#)
- Lista de funciones permitidas [acá](#)
- Para testear el código pueden usar [este](#) archivo que ya cuenta con todo lo necesario para desarrollar sus propios tests (este archivo no se entrega)

Implementar las siguientes especificaciones en Python.

Ejercicio 1 [2 puntos]

Implementar la función cantidad_parejas_que_suman:

problema cantidad_parejas_que_suman (in $s: \text{seq}\langle Z \rangle$, in $n: Z$) : Z {
 requiere: { - }
 asegura: { res es la cantidad de parejas $s[i]$ y $s[j]$ de números de s tales que $s[i] + s[j] = n$ (con $i < j$) }
}

Ejemplo: cantidad_parejas_que_suman([1,3,2,5,4,8], 5) debe devolver 2

Ejercicio 2 [2,5 puntos]

En un supermercado tenemos una fila de clientes esperando para ser atendidos por algún cajero. Cada cliente tiene un nombre, un método de pago y una cantidad de productos. La fila de clientes se representa como una Cola de $\text{String} \times \text{String} \times \mathbb{Z}$, donde el primer elemento es el nombre del cliente, el segundo es el método de pago y el tercero es la cantidad de productos. Implementar la función `pasar_por_autoservicio`:

Nota: los métodos de pago son strings conformados por letras minúsculas.

```
problema pasar_por_autoservicio (inout clientes: Cola( String x String x Z )) : String {  
  requiere: { Las primeras componentes de clientes son strings no vacíos y todos distintos entre sí }  
  requiere: { Las terceras componentes de clientes son números positivos }  
  requiere: { Existe al menos un elemento c dentro de la cola clientes tal que  $c_1 \neq \text{"efectivo"}$  y  $c_2 \leq 15$  }  
  modifica: { clientes }  
  asegura: { Sea c el primer elemento insertado en la cola clientes tal que  $c_1 \neq \text{"efectivo"}$  y  $c_2 \leq 15$ , entonces  $res = c_0$  }  
  asegura: { clientes contiene todos los elementos de clientes@pre excepto la tupla que contiene a res en su primera posición, en el mismo orden que en clientes@pre. }  
}
```

Ejemplo: `pasar_por_autoservicio(clientes)` debe devolver **"Bruno"** (y quitar su tupla de la cola) si *clientes* es una cola en la cual se insertaron (en orden) los siguientes elementos:

1. ("Ana", "efectivo", 13)
2. ("Juan", "qr", 22)
3. ("Bruno", "tarjeta", 14)

3) Ejercicio 3 [2,5 puntos]

Implementar la función `intercambiar_e_invertir_columnas`:

```
problema intercambiar_e_invertir_columnas (inout A: seq(seq(Z)), in col1: Z, in col2: Z) {  
  requiere: { Todas las filas de A tienen la misma longitud (estrictamente positiva) }  
  requiere: {  $|A| > 0$  }  
  requiere: {  $0 \leq col1 < |A[0]|$  }  
  requiere: {  $0 \leq col2 < |A[0]|$  }  
  requiere: {  $col1 \neq col2$  }  
  modifica: { A }  
  asegura: { A tiene exactamente las mismas dimensiones que A@pre }  
  asegura: {  $A[i][j] = A@pre[i][j]$  para todo  $i, j$  en rango tal que  $j \neq col1$  y  $j \neq col2$  }  
  asegura: {  $A[i][col1] = A@pre[|A|-1-i][col2]$  para todo  $i$  tal que  $0 \leq i < |A|$  }  
  asegura: {  $A[i][col2] = A@pre[|A|-1-i][col1]$  para todo  $i$  tal que  $0 \leq i < |A|$  }  
}
```

Ejemplo: Si `mat = [[1,2,3],[40,50,60], [-7,-8,-9]]`, luego de ejecutarse `intercambiar_e_invertir_columnas(mat,1,2)` debería ocurrir que `print(mat)` muestre `[[1, -9, -8], [40, 60, 50], [-7, 3, 2]]`

Ejercicio 4 [2 puntos]

Se realizaron dos censos en los cuales se le preguntó a cada persona en que localidad vive. Estos datos fueron almacenados en dos diccionarios cuyas claves son los nombres de las personas, y sus valores las localidades en las cuales viven. Implementar la función `mantuvieron_residencia`:

problema `mantuvieron_residencia` (in *censo1*: Diccionario<String,String>, in *censo2*: Diccionario<String,String>): Diccionario<String,Z> {
 requiere: { Las claves de *censo1* son las mismas que las claves de *censo2* }
 asegura: { *k* es clave de *res* si y sólo si existe alguna clave *p* en *censo1* tal que al obtener su valor tanto en *censo1* como en *censo2*, este es igual a *k* }
 asegura: { El valor de cada clave de *res* representa la cantidad de personas que en ambos censos vivía en esa localidad, es decir, que mantuvieron su residencia en la misma localidad entre ambos censos }
}

Ejemplo: `mantuvieron_residencia({'Juan': 'Merlo', 'Ana': 'Merlo'}, {'Juan': 'Castelar', 'Ana': 'Merlo'})`
debe devolver `{'Merlo': 1}`

Ejercicio 5 [0,5 puntos]

Dada la siguiente especificación y una posible implementación de la misma, conteste marcando la opción correcta.

<pre>problema sumar_elementos (in s: seq<Z>): Z { requiere: { - } asegura: { res es la suma de los elementos de s } }</pre>	<pre>def sumar_elementos(s: list[int]) -> int: res: int = 0 for i in range(1, len(s)): res += s[i] return res</pre>
---	--

- ☐ El código es correcto, calcula lo pedido en la especificación para cualquier input
- ☐ El código tiene un bug, y si hacemos un test suite que cubra todas las líneas lo detectaremos
- ☐ El código tiene un bug, pero es posible hacer un test suite que cubra todas las líneas y no detectar dicho bug

Ejercicio 6 [0,5 puntos]

Supongamos que un programa tiene un ciclo que itera sobre todos los elementos de una lista de tamaño *n*. Si el programa realiza una operación constante dentro de ese ciclo (por ejemplo, incrementa en 1 cada valor de la lista), ¿cómo afecta el tamaño de la lista al número de operaciones?

- ☐ Si la lista tiene más elementos, el número de operaciones aumentará.
- ☐ El número de operaciones no cambia con el tamaño de la lista.
- ☐ El número de operaciones depende del contenido de la lista, no del tamaño.

Adjunta el archivo con tu solución:

Solo se puede adjuntar 1 archivo de extensión .py. En caso de haber desarrollado tests propios, no deben ser entregados.

Seleccionar archivo a enviar: Ningún archivo seleccionado

Enviar

Solucion entregada por el alumno

```
from queue import Queue as Cola
```

```
'''
```

```
Ejercicio 1 [2 puntos]
```

```
Implementar la función cantidad_parejas_que_suman:
```

```
problema cantidad_parejas_que_suman (in s: seq(Z), in n: Z) : Z {  
  requiere: { - }  
  asegura: { res es la cantidad de parejas s[i] y s[j] de números de s tales que s[i] + s[j] = n (con i < j) }  
}  
'''
```

```
# Ejercicio 1
```

```
def cantidad_parejas_que_suman(s: list[int], n: int) -> int:  
    cantidad_parejas: int = 0  
    longitud_secuencia: int = len(s)
```

```
    for indice in range(0, longitud_secuencia):  
        numero_actual = s[indice]
```

```
        indice_mayor = indice + 1
```

```
        for indice_posterior in range(indice_mayor, longitud_secuencia):  
            numero_posterior = s[indice_posterior]  
            suma_numeros = numero_actual + numero_posterior  
            if suma_numeros == n:  
                cantidad_parejas += 1
```

```
    return cantidad_parejas
```

```
'''
```

```
Ejercicio 2 [2,5 puntos]
```

En un supermercado tenemos una fila de clientes esperando para ser atendidos por algún cajero. Cada cliente tiene un nombre, un método de pago y una cantidad de productos.

La fila de clientes se representa como una Cola de String x String x Z, donde el primer elemento es el nombre del cliente, el segundo es el método de pago y el tercero es la cantidad de productos.

Implementar la función pasar_por_autoservicio:

Nota: los métodos de pago son strings conformados por letras minúsculas.

```
problema pasar_por_autoservicio (inout clientes: Cola( String x String x Z )) : String {  
  requiere:{ Las primeras componentes de clientes son strings no vacíos y todos distintos entre sí }  
  requiere:{ Las terceras componentes de clientes son números positivos }  
  requiere:{ Existe al menos un elemento c dentro de la cola clientes tal que c1 ≠ "efectivo" y c2 ≤ 15 }  
  modifica: { clientes }  
  asegura: { Sea c el primer elemento insertado en la cola clientes tal que c1 ≠ "efectivo" y c2 ≤ 15, entonces res = c0 }  
  asegura: { clientes contiene todos los elementos de clientes@pre excepto la tupla que contiene a res en su primera posición, en el mismo orden que en clientes@pre. }
```

```

}
...

# Ejercicio 2
def pasar_por_autoservicio(clientes: Cola[tuple[str, str, int]]) -> str:

    clientes_nueva: Cola[tuple[str, str, int]] = Cola()
    cliente_prioridad: tuple[str, str, int] = (""," ",0)

    while (not clientes.empty()) and cliente_prioridad == (""," ",0):
        cliente_actual: tuple[str, str, int] = clientes.get()

        if cliente_actual[1] != "efectivo" and cliente_actual[2] <= 15:
            cliente_prioridad = cliente_actual

        else:
            clientes_nueva.put(cliente_actual)

    while not clientes.empty():
        clientes_nueva.put(clientes.get())

    ''' CREO QUE PARA ARREGLAR LOS ERRORES ALCANZARIA CON UN WHILE QUE DIGA:
    while not clientes_nueva.empty():
        clientes.put(clientes_nueva.get())
    TAL QUE PONGAMOS TODOS LOS CLIENTES QUE PUSIMOS EN UNA COLA AUXILIAR ("cola_nueva") EN LA COLA ORIGINAL'''

    clientes = clientes_nueva # Y Esto es lo que esta mal xD

    return cliente_prioridad[0]

'''
3) Ejercicio 3 [2,5 puntos]

```

Implementar la función intercambiar_e_invertir_columnas:

```

problema intercambiar_e_invertir_columnas(inout A: seq(seq(Z)), in col1: Z, in col2: Z) {
    requiere: { Todas las filas de A tienen la misma longitud (estrictamente positiva) }
    requiere: { |A| > 0 }
    requiere: { 0 ≤ col1 < |A[0]| }
    requiere: { 0 ≤ col2 < |A[0]| }
    requiere: { col1 ≠ col2 }
    modifica: { A }
    asegura: { A tiene exactamente las mismas dimensiones que A@pre }
    asegura: { A[i][j] = A@pre[i][j] para todo i, j en rango tal que j ≠ col1 y j ≠ col2 }
    asegura: { A[i][col1] = A@pre[|A|-1-i][col2] para todo i tal que 0 ≤ i < |A| }
    asegura: { A[i][col2] = A@pre[|A|-1-i][col1] para todo i tal que 0 ≤ i < |A| }
}
'''

```

```

# Ejercicio 3
def intercambiar_e_invertir_columnas(A: list[list[int]], col1: int, col2: int) -> None:
    longitud_matriz: int = len(A)
    longitud_fila: int = len(A[0])

    nueva_matriz: list[list[int]] = []

    for indice_fila in range(0, longitud_matriz):
        nueva_fila: list[int] = []

```

```

    for indice_columna in range(0, longitud_fila):
        if indice_columna == col1:
            nueva_fila.append(A[longitud_matriz - 1 - indice_fila][col2])
        elif indice_columna == col2:
            nueva_fila.append(A[longitud_matriz - 1 - indice_fila][col1])
        else:
            nueva_fila.append(A[indice_fila][indice_columna])

    nueva_matriz.append(nueva_fila)

A.clear()

for fila in nueva_matriz:
    fila_nueva = []
    for columna in fila:
        fila_nueva.append(columna)
    A.append(fila_nueva)

return

```

'''
Ejercicio 4 [2 puntos]

Se realizaron dos censos en los cuales se le preguntó a cada persona en que localidad vive. Estos datos fueron almacenados en dos diccionarios cuyas claves son los nombres de las personas, y sus valores las localidades en las cuales viven. Implementar la función mantuvieron_residencia:

```

problema mantuvieron_residencia (in censo1: Diccionario(String,String), in censo2: Diccionario(String,String)): Diccionario(String,Z) {
    requiere: { Las claves de censo1 son las mismas que las claves de censo2 }
    asegura: { k es clave de res si y sólo si existe alguna clave p en censo1 tal que al obtener su valor tanto en censo1 como en censo2, este es igual a k }
    asegura: { El valor de cada clave de res representa la cantidad de personas que en ambos censos vivía en esa localidad, es decir,
    que mantuvieron su residencia en la misma localidad entre ambos censos }
}
'''

```

Ejercicio 4

```

def mantuvieron_residencia(censo1: dict[str, str], censo2: dict[str, str]) -> dict[str, int]:
    diccionario_municipios: dict[str, int] = {}

```

```

    for valor in censo1.values():
        if valor not in diccionario_municipios.keys():
            diccionario_municipios[valor] = 0

    for clave in censo1.keys():
        municipio_anterior: str = censo1[clave]
        municipio_actual: str = censo2[clave]

        if municipio_anterior == municipio_actual:
            diccionario_municipios[municipio_actual] += 1

```

```

    diccionario_filtrado: dict[str, int] = {} #Falta filtrar que ninguna clave tenga valor 0 para cumplir el primer asegura

```

```

    for clave, valor in diccionario_municipios.items():
        if valor != 0:
            diccionario_filtrado[clave]= valor

```

```
return diccionario_filtrado
```

Resultado de la compilacion

Ejecucion de los tests

tema2-test-ej1.py.compilacion.out

Puntaje del ej: 2 / 2

```
test_lista_vacia_devuelve_cero (__main__.Ej1Test.test_lista_vacia_devuelve_cero) ... ok
test_no_hay_parejas (__main__.Ej1Test.test_no_hay_parejas) ... ok
test_no_hay_parejas_con_si_mismo (__main__.Ej1Test.test_no_hay_parejas_con_si_mismo) ... ok
test_parejas_con_numeros_negativos (__main__.Ej1Test.test_parejas_con_numeros_negativos) ... ok
test_parejas_repetidas_cuenta_todas (__main__.Ej1Test.test_parejas_repetidas_cuenta_todas) ... ok
test_parejas_suman_valores_varios (__main__.Ej1Test.test_parejas_suman_valores_varios) ... ok
test_suman_cero (__main__.Ej1Test.test_suman_cero) ... ok
test_todos_forman_pareja (__main__.Ej1Test.test_todos_forman_pareja) ... ok
test_todos_negativos (__main__.Ej1Test.test_todos_negativos) ... ok
test_un_elemento_no_puede_formar_pareja (__main__.Ej1Test.test_un_elemento_no_puede_formar_pareja) ... ok
```

```
-----
Ran 10 tests in 0.005s
```

OK

tema2-test-ej2.py.compilacion.out

Puntaje del ej: 1.75 / 2.5

```
test_cliente_al_final_de_laCola_se_retira (__main__.Ej2Test.test_cliente_al_final_de_laCola_se_retira) ... ok
test_cliente_al_inicio_de_laCola_se_retira (__main__.Ej2Test.test_cliente_al_inicio_de_laCola_se_retira) ... ok
test_cliente_al_medio_de_laCola_se_retira (__main__.Ej2Test.test_cliente_al_medio_de_laCola_se_retira) ... ok
test_cliente_al_medio_de_laCola_se_retira_y_el_resto_quedan (__main__.Ej2Test.test_cliente_al_medio_de_laCola_se_retira_y_el_resto_quedan) ... FAIL
test_selecciona_al_primer_cliente_con_todos_candidatos (__main__.Ej2Test.test_selecciona_al_primer_cliente_con_todos_candidatos) ... ok
test_selecciona_al_primer_cliente_con_todos_candidatos_y_el_resto_quedan (__main__.Ej2Test.test_selecciona_al_primer_cliente_con_todos_candidatos_y_el_resto_quedan) ... FAIL
test_selecciona_al_primer_cliente_con_varios_candidatos (__main__.Ej2Test.test_selecciona_al_primer_cliente_con_varios_candidatos) ... ok
test_selecciona_al_primer_cliente_con_varios_candidatos_y_el_resto_quedan (__main__.Ej2Test.test_selecciona_al_primer_cliente_con_varios_candidatos_y_el_resto_quedan) ... FAIL
test_unico_cliente_se_retira (__main__.Ej2Test.test_unico_cliente_se_retira) ... ok
test_unico_cliente_se_retira_yCola_queda_vacia (__main__.Ej2Test.test_unico_cliente_se_retira_yCola_queda_vacia) ... ok
```

```
=====
FAIL: test_cliente_al_medio_de_laCola_se_retira_y_el_resto_quedan (__main__.Ej2Test.test_cliente_al_medio_de_laCola_se_retira_y_el_resto_quedan)
-----
```

Traceback (most recent call last):

```
File "/env/lib/python3.12/site-packages/timeout_decorator/timeout_decorator.py", line 82, in new_function
    return function(*args, **kwargs)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/correcciones/parcial-python-tm/tema2-test-ej2.py", line 59, in test_cliente_al_medio_de_laCola_se_retira_y_el_resto_quedan
    self.assertEqual(clientes.queue, clientes_final.queue)
```

```

AssertionError: deque([]) != deque([('Juan', 'tarjeta', 22), ('Ana', 'efectivo', 13)])

=====
FAIL: test_selecciona_al_primer_cliente_con_todos_candidatos_y_el_resto_quedan (__main__.Ej2Test.test_selecciona_al_primer_cliente_con_todos_candidatos_y_el_resto_quedan)
-----
Traceback (most recent call last):
  File "./env/lib/python3.12/site-packages/timeout_decorator/timeout_decorator.py", line 82, in new_function
    return function(*args, **kwargs)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "./correcciones/parcial-python-tm/tema2-test-ej2.py", line 87, in test_selecciona_al_primer_cliente_con_todos_candidatos_y_el_resto_quedan
    self.assertEqual(clientes.queue, clientes_final.queue)
AssertionError: deque([]) != deque([('Pedro', 'transferencia', 12), ('Ana', '[31 chars]14)])

=====
FAIL: test_selecciona_al_primer_cliente_con_varios_candidatos_y_el_resto_quedan (__main__.Ej2Test.test_selecciona_al_primer_cliente_con_varios_candidatos_y_el_resto_quedan)
-----
Traceback (most recent call last):
  File "./env/lib/python3.12/site-packages/timeout_decorator/timeout_decorator.py", line 82, in new_function
    return function(*args, **kwargs)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "./correcciones/parcial-python-tm/tema2-test-ej2.py", line 73, in test_selecciona_al_primer_cliente_con_varios_candidatos_y_el_resto_quedan
    self.assertEqual(clientes.queue, clientes_final.queue)
AssertionError: deque([]) != deque([('Juan', 'qr', 22), ('Pedro', 'transferen[57 chars]14)])

-----
Ran 10 tests in 0.008s

FAILED (failures=3)

```

tema2-test-ej3.py.compilacion.out

Puntaje del ej: 2.5 / 2.5

```

test_intercambiar_e_invertir_columnas_2x3 (__main__.Ej1Test.test_intercambiar_e_invertir_columnas_2x3) ... ok
test_intercambiar_e_invertir_columnas_3x2 (__main__.Ej1Test.test_intercambiar_e_invertir_columnas_3x2) ... ok
test_intercambiar_e_invertir_columnas_con_ceros (__main__.Ej1Test.test_intercambiar_e_invertir_columnas_con_ceros) ... ok
test_intercambiar_e_invertir_columnas_con_negativos (__main__.Ej1Test.test_intercambiar_e_invertir_columnas_con_negativos) ... ok
test_intercambiar_e_invertir_columnas_en_matriz_1x2 (__main__.Ej1Test.test_intercambiar_e_invertir_columnas_en_matriz_1x2) ... ok
test_intercambiar_e_invertir_columnas_en_matriz_2x2 (__main__.Ej1Test.test_intercambiar_e_invertir_columnas_en_matriz_2x2) ... ok
test_intercambiar_e_invertir_columnas_en_matriz_3x10 (__main__.Ej1Test.test_intercambiar_e_invertir_columnas_en_matriz_3x10) ... ok
test_intercambiar_e_invertir_columnas_en_matriz_3x3 (__main__.Ej1Test.test_intercambiar_e_invertir_columnas_en_matriz_3x3) ... ok
test_intercambiar_e_invertir_columnas_en_matriz_8x3 (__main__.Ej1Test.test_intercambiar_e_invertir_columnas_en_matriz_8x3) ... ok
test_intercambiar_e_invertir_columnas_iguales (__main__.Ej1Test.test_intercambiar_e_invertir_columnas_iguales) ... ok

-----
Ran 10 tests in 0.008s

OK

```

tema2-test-ej4.py.compilacion.out

Puntaje del ej: 2 / 2

```

test_no_modifica_inputs (__main__.Ej1Test.test_no_modifica_inputs) ... ok
test_residencia_mantenida_en_una_localidad (__main__.Ej1Test.test_residencia_mantenida_en_una_localidad) ... ok

```

```
test_residencia_mantenida_en_varias_localidades (__main__.Ej1Test.test_residencia_mantenida_en_varias_localidades) ... ok
test_residencias_cruzadas_no_mantenidas (__main__.Ej1Test.test_residencias_cruzadas_no_mantenidas) ... ok
test_sin_habitantes (__main__.Ej1Test.test_sin_habitantes) ... ok
test_sin_residencia_mantenida (__main__.Ej1Test.test_sin_residencia_mantenida) ... ok
test_todos_mantienen_residencia (__main__.Ej1Test.test_todos_mantienen_residencia) ... ok
test_todos_se_mudan (__main__.Ej1Test.test_todos_se_mudan) ... ok
test_una_sola_persona_mantiene (__main__.Ej1Test.test_una_sola_persona_mantiene) ... ok
test_una_sola_persona_se_muda (__main__.Ej1Test.test_una_sola_persona_se_muda) ... ok
```

Ran 10 tests in 0.007s

OK

mchoice.json-ej5.compilacion.out

Puntaje del ej: 0.0 / 0.5

mchoice ej5: respuesta del alumno=2, respuesta correcta=3

Ran 1 test in 0 seconds

FAILED (failures=1)

mchoice.json-ej6.compilacion.out

Puntaje del ej: 0.5 / 0.5

mchoice ej6: respuesta del alumno=1, respuesta correcta=1

Ran 1 test in 0 seconds

OK

FIN